

TP10 — Simuler un déploiement réseau sans coupure (ZDD)

Formation NetOps : Automatiser la gestion de son Infrastructure Réseau — Ambient IT

Description

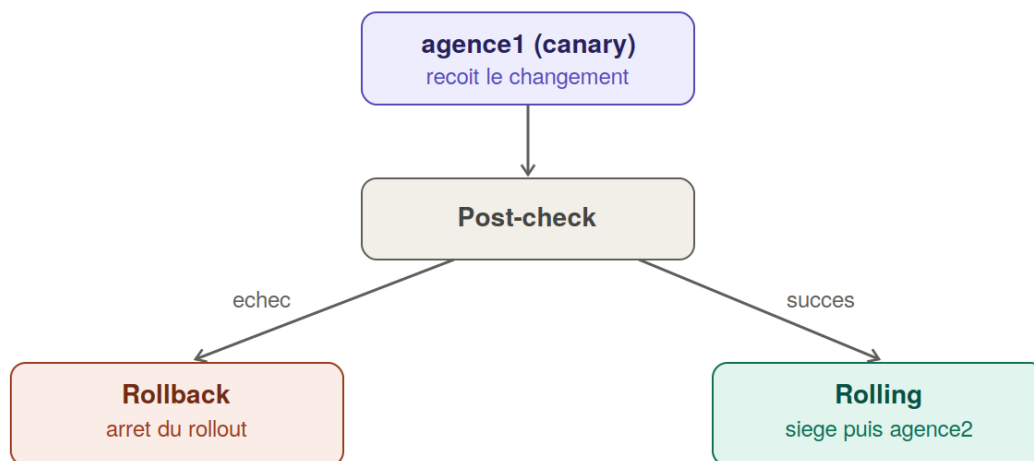


Figure 1: Déploiement ZDD : canary puis rolling ou rollback

Ce TP transforme le déploiement de configuration (TP7) en un déploiement sans coupure : un site pilote (canary) reçoit le changement en premier et passe des post-checks ; si tout est bon, le reste des sites est déployé un par un (rolling) ; si le canary échoue, la configuration précédente est restaurée automatiquement et aucun autre site n'est touché.

Prérequis

- TP1 à TP9 validés

Étapes

Étape 1 — Créer sa branche

```
cd netops-project-<nom-groupe>
git checkout dev
git pull origin dev
git checkout -b <prenom>-<nom>-jour3-tp10
```

Étape 2 — Écrire le playbook de déploiement ZDD

```
cat > playbooks/deploy_zdd.yml << 'EOF'
---
- name: Déploiement sans coupure (canary puis rolling) des configurations reseau
  hosts: agencel,siege,agence2
  gather_facts: no
  serial:
    - 1
    - 1
    - 1
  any_errors_fatal: true

  tasks:
    - name: "Sauvegarder et deployer la configuration ({{ inventory_hostname }})"
      ansible.builtin.template:
        src: "{{ playbook_dir }}/../templates/interface_config.j2"
        dest: "{{ playbook_dir }}/../generated_configs/{{ inventory_hostname }}.cfg"
        backup: yes
        register: render_result

    - name: "Post-check et rollback automatique en cas d'echec ({{ inventory_hostname }})"
      block:
        - name: "Verifier que la configuration generee respecte les invariants attendus"
          ansible.builtin.assert:
            that:
              - "'ntp server' in lookup('file', playbook_dir + '/../generated_configs/' + inventory_hostname + '.cfg')"
              - "'banner motd' in lookup('file', playbook_dir + '/../generated_configs/' + inventory_hostname + '.cfg')"
              - "'hostname ' + inventory_hostname in lookup('file', playbook_dir + '/../generated_configs/' + inventory_hostname + '.cfg'"
            fail_msg: "Post-check echoue sur {{ inventory_hostname }} : configuration invalide detectee"
            success_msg: "Post-check reussi sur {{ inventory_hostname }}"

      rescue:
        - name: "ROLLBACK : restaurer la configuration precedente sur {{ inventory_hostname }}"
          ansible.builtin.copy:
            src: "{{ render_result.backup_file }}"
            dest: "{{ playbook_dir }}/../generated_configs/{{ inventory_hostname }}.cfg"
            remote_src: yes
            when: render_result.backup_file is defined
```

```
- name: "Arreter tout le deploiement suite a l'echec sur {{ inventory_hostname }}"
  ansible.builtin.fail:
```

```
msg: "Deploiement ZDD arrete : {{ inventory_hostname }} a echoue les post-checks. Rollback effectue. Les sites suivants ne seront PAS modifies."
```

EOF

Trois mécanismes clés dans ce playbook : - hosts: agence1,siege,agence2 — ordre explicite (agence1 en premier, désigné comme site canary) - serial: [1, 1, 1] — traite les hôtes un par un, dans cet ordre, plutôt que tous en même temps (le comportement par défaut d'Ansible) - any_errors_fatal: true — si un hôte échoue dans un batch, tout le reste du déploiement s'arrête immédiatement, même les hôtes qui n'ont pas encore été traités

Étape 3 — Tester le déroulement normal (sans panne)

```
cd lab && docker compose up -d --build && cd ..
ansible-playbook --vault-password-file .vault_pass playbooks/deploy_zdd.yml
```

Résultat attendu : 3 blocs PLAY s'affichent successivement (un par hôte, dans l'ordre agence1 → siege → agence2), chacun avec son déploiement et son post-check réussis.

Étape 4 — Simuler un changement défectueux (bug introduit par erreur)

```
cp templates/interface_config.j2 /tmp/interface_config_backup.j2
sed -i '/ntp server/d' templates/interface_config.j2
```

On vient de casser volontairement le template — la ligne ntp server disparaît de toute configuration générée, exactement comme un vrai bug qu'un développeur pourrait introduire par erreur.

Étape 5 — Lancer le déploiement avec le bug en place

```
ansible-playbook --vault-password-file .vault_pass playbooks/deploy_zdd.yml
```

Résultat attendu :

- Un seul bloc PLAY s'affiche (celui d'agence1, le canary) — aucun bloc pour siege ou agence2, le rollout ne va pas plus loin
- La tâche de post-check échoue sur agence1 (ntp server absent)
- Le rescue s'active : ROLLBACK : restaurer la configuration precedente sur agence1
- Le message final : Deploiement ZDD arrete : agence1 a echoue les post-checks. Rollback effectue. Les sites suivants ne seront PAS modifies.
- Récapitulatif : agence1 : ... failed=1 rescued=1, et rien du tout pour siege/agence2 (ils n'apparaissent même pas, puisqu'ils n'ont jamais été atteints)

Étape 6 — Vérifier le rollback et l'absence d'impact sur les autres sites

```
cat generated_configs/agence1.cfg
```

Doit contenir de nouveau `ntp server 10.0.0.1` — la version précédente a bien été restaurée malgré le bug.

```
grep "ntp" generated_configs/siege.cfg
grep "ntp" generated_configs/agence2.cfg
```

Les deux doivent toujours contenir `ntp server 10.0.0.1`, preuve qu'ils n'ont subi aucune modification.

Étape 7 — Réparer le template

```
cp /tmp/interface_config_backup.j2 templates/interface_config.j2
grep "ntp" templates/interface_config.j2
```

Doit réafficher `ntp server {{ ntp_server }}`. Relancez une dernière fois pour confirmer que tout redevient normal :

```
ansible-playbook --vault-password-file .vault_pass playbooks/deploy_zdd.yml
```

Étape 8 — Committer et pousser

```
git add playbooks/deploy_zdd.yml
git commit -m "TP10 : déploiement ZDD (canary puis rolling, rollback automatique)"
git push -u origin <prenom>-<nom>-jour3-tp10
```

Ouvrir une pull request <prenom>-<nom>-jour3-tp10 → dev.

Comprendre le TP

Canary deployment : au lieu de déployer un changement partout en même temps (et de découvrir un problème sur tous les sites à la fois), on le déploie d'abord sur un seul site pilote. S'il échoue ses post-checks, l'impact est limité à un site — exactement le principe qui, dans le scénario NexaCorp du départ, aurait évité que l'ingénieur découvre un problème sur l'ensemble du parc après un déplacement de 2 jours.

Rolling update : une fois le canary validé, les sites restants sont mis à jour un par un (`serial: 1`), jamais tous simultanément — ça limite l'impact d'un problème qui ne serait apparu qu'après plusieurs déploiements, et ça laisse le temps d'observer entre chaque site.

any_errors_fatal: true est ce qui transforme une suite de batches indépendants en un vrai pipeline "tout s'arrête si le premier échoue" : sans ce réglage, un échec sur `agence1` n'empêcherait pas Ansible de continuer vers `siege` et `agence2` dans leurs propres batches — avec ce réglage, l'échec du canary annule tout le reste du rollout, batches non commencés inclus.

backup: yes + rollback dans le rescue : le module `template` sait automatiquement sauvegarder l'ancien fichier avant de le remplacer (`render_result.backup_file` contient le chemin de cette sauvegarde). En cas d'échec du post-check, le `rescue` restaure cette sauvegarde — la configuration revient exactement à son état d'avant tentative, sans intervention manuelle.

Validation

- ☐ Déploiement normal : 3 sites traités un par un, dans l'ordre agence1 → siege → agence2
- ☐ Avec un bug volontaire dans le template, seul agence1 (canary) est traité — siege et agence2 ne sont jamais atteints
- ☐ Le rollback restaure correctement agence1.cfg à son état précédent
- ☐ siege.cfg et agence2.cfg restent inchangés pendant l'échec du canary
- ☐ Une pull request <prenom>-<nom>- jour3-tp10 → dev est ouverte